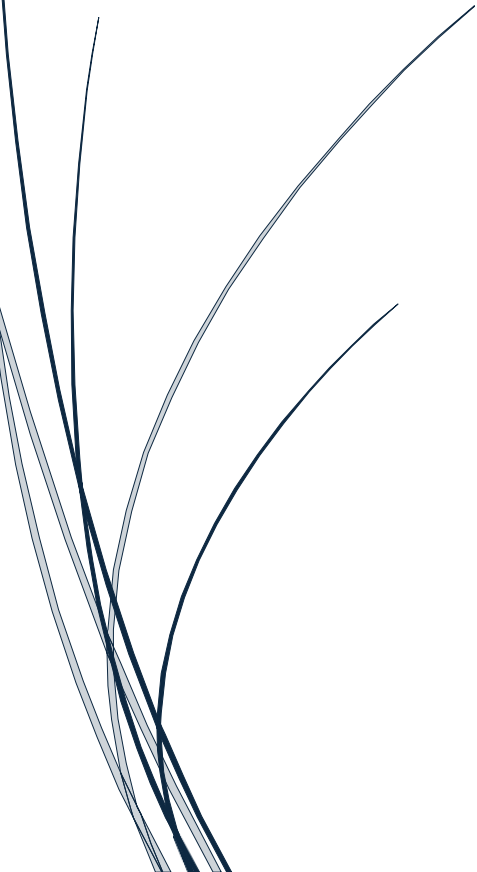


11/16/2025

Bank Marketing Project Report

Author: Mohamad Badra



Introduction

This project focuses on analyzing a bank marketing dataset to understand which customers are more likely to subscribe to a term deposit. The data includes information about clients such as their age, account balance, the duration of previous marketing calls, and campaign details. Following data modeling steps — preprocessing, exploratory analysis, and building the classification model — our aim is to identify the factors that influence customer decisions in order to build a model that can predict subscription outcomes accurately. This helps improve targeting strategies and supports better decision-making in future marketing campaigns.

Project Phases

Data Collection

The dataset for this project was taken from the UCI Machine Learning Repository, a well-known source for high quality, machine-learning-related datasets. It contains real information from a Portuguese bank's marketing campaign, where the bank called clients to ask if they would subscribe to a term deposit. In this phase, we obtained the dataset, loaded it properly on our IDE, and made sure all columns and records were ready for the project's next steps.

Dataset link: [UCI Machine Learning Repository](#)

Used Data Name: bank-full.csv

Discovery For Data

We started by checking the shape of the dataset to see the total number of rows and columns. Next, we checked the data types of all columns, then listed the names of numeric and categorical columns to handle them separately. We reviewed the statistical summary to understand the distribution of the numeric variables, then described the dataset in more detail, including the number of columns, rows, and the kinds of values stored in each feature. We checked for missing values, unique values, and the values each column consists of, examined the target variable distribution (imbalance check), and — for convenience — converted month names to month numbers.

EDA (Exploratory Data Analysis)

During the EDA phase, we first visualized the class imbalance in the target variable to understand how imbalanced the "yes" and "no" responses were. We then examined the distributions of the main numeric variables. For the categorical features, we visualized the top categories in each column to see which values were most common, and compared all categorical variables with the target y using bar charts, which helped us observe how each category relates to subscribing to a term deposit.

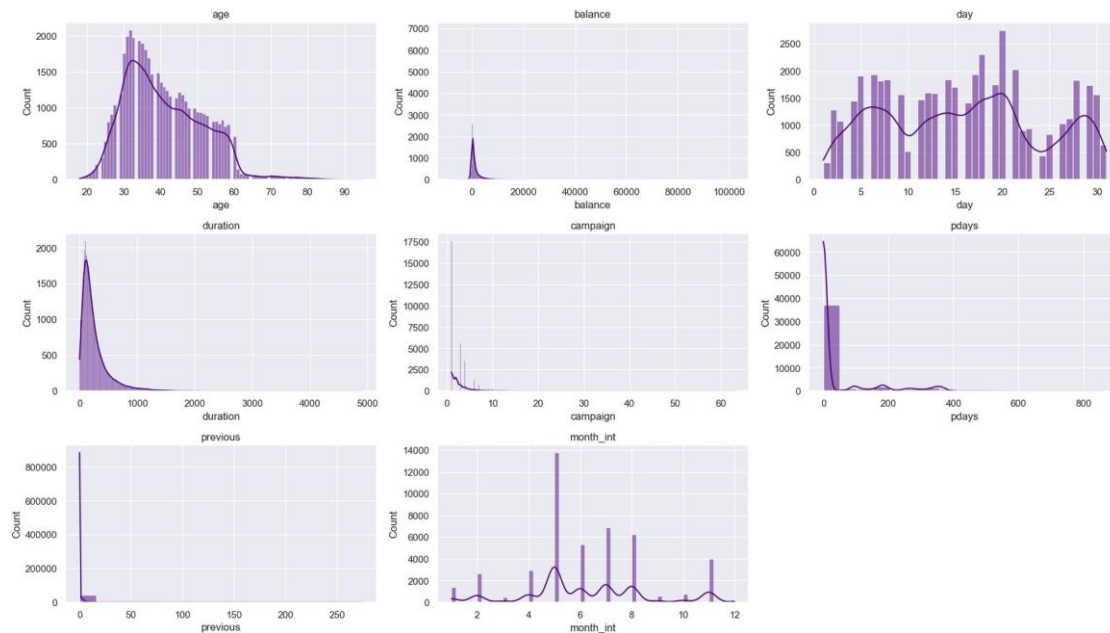


Figure 1. Distribution of the numeric features (age, balance, day, duration, campaign, pdays, previous, month) and the resulting class imbalance in the target variable.

Data Cleaning

In the data cleaning phase, we first handled the missing values. Since this dataset represents missing information using the label "unknown", we started by counting these values to understand whether they appeared randomly or followed a pattern. The results showed no meaningful pattern, so each column with "unknown" was treated independently. Some "unknown" values were replaced with "other" when they simply represented an unspecified category. For the poutcome column, the value "unknown" actually means the client was never contacted in a previous campaign, so we replaced it with a more meaningful label, "nonexistent", to avoid confusion and improve the upcoming processing.

In the next step, we identified outliers using boxplots, which showed that the balance column had very high values on the upper end that could negatively affect modeling. Based on the age distribution shape and IQR, the reasonable upper limit was around 3,500, but because bank balances naturally vary among clients, we used a more flexible threshold of 5,000. To avoid losing the already small number of positive ("yes") cases, we removed only the extreme outliers where the client did not subscribe, and kept all records where the balance was high but the client subscribed. This approach reduces noise without harming model performance on the minority class.

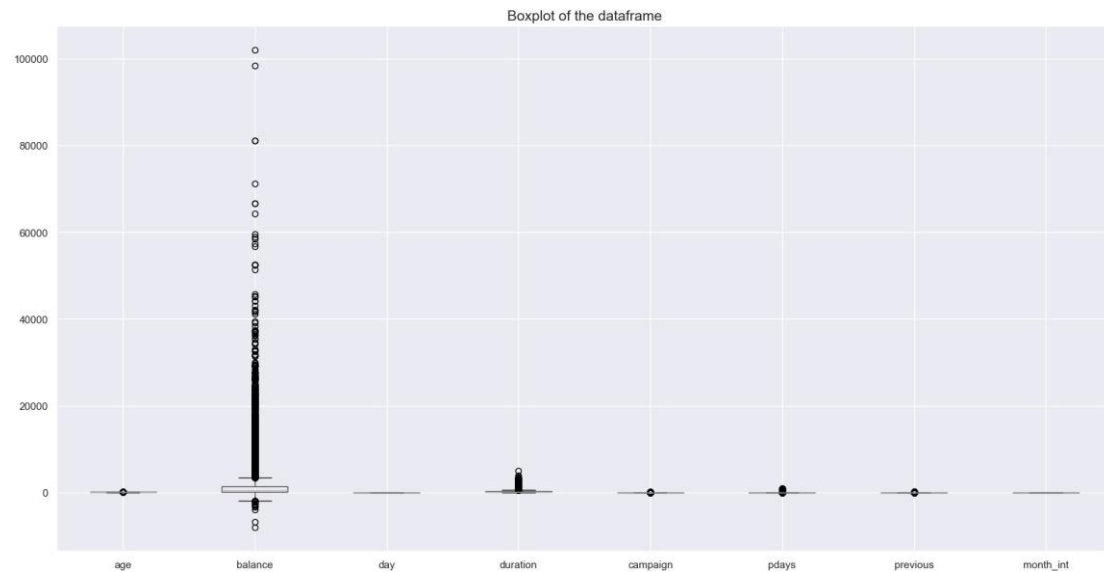


Figure 2. Boxplot of all numeric columns before cleaning – balance shows extreme high-end outliers (up to 102,127) relative to the rest of the distribution.

Feature Engineering / Encoding

Feature engineering involves creating or transforming features to help the model understand the data better and improve its performance. In this project, we applied a logical transformation to the day column. Since the days of the month (1–30) do not have a meaningful ordinal weight, using them directly can negatively affect the model. To fix this, we converted the day numbers into week categories, assuming a simple structure where a month has 4 weeks:

- Days 1–7 → Week 1
- Days 8–14 → Week 2
- Days 15–21 → Week 3
- Days 22–31 → Week 4

This transformation allows the model to understand which part of the month the contact happened, instead of treating every day as a separate numeric value.

Feature Encoding

Computers cannot understand text like humans, so categorical columns must be converted into numeric values through feature encoding. This step allows the model to interpret and compare different categories correctly. In our dataset, we applied encoding to several columns such as marital, education, housing, loan, and the target variable y, converting their categories into numeric labels like 0, 1, 2, or 3.

Additionally, we removed an unimportant feature that was no longer needed after creating its alternative. This keeps the dataset cleaner and helps the model focus on the most useful information.

Correlation and VIF Calculation

Correlation helps us understand how strongly two features are related to each other, which helps the model in prediction. To visualize these relationships, we created a correlation heatmap, which shows how much each numeric variable is connected to the others.

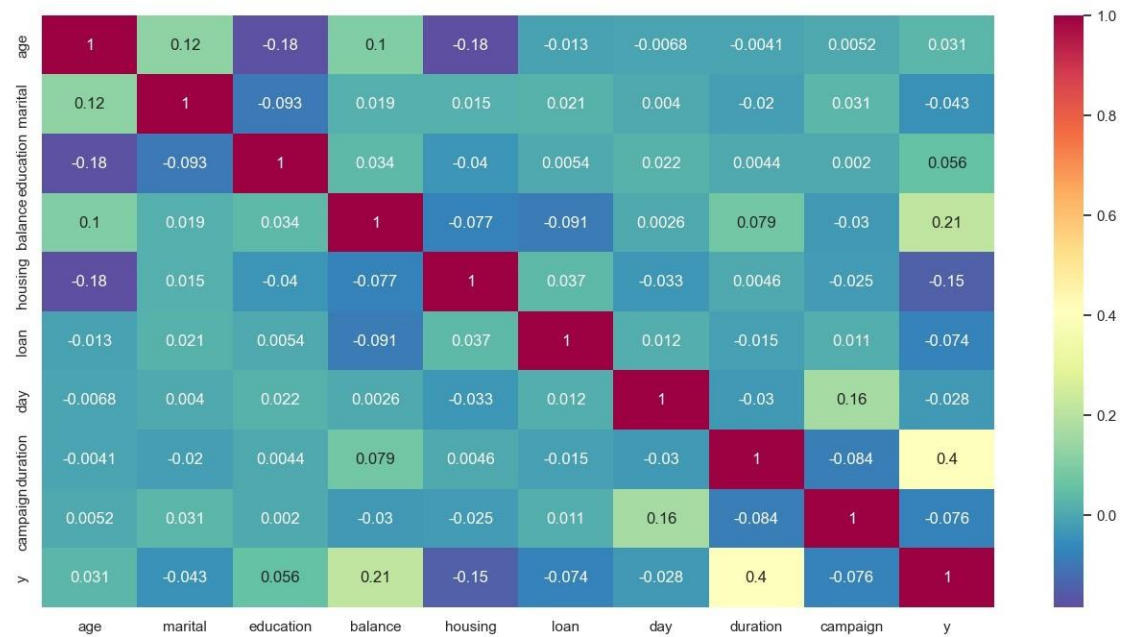


Figure 3. Correlation heatmap of the numeric features and the target variable y .

We also calculated the Variance Inflation Factor (VIF). VIF values help us see whether a feature is too dependent on the rest, and check for multicollinearity, which happens when one feature can be predicted from others. Although the general rule is to consider a VIF above 5 as high, this threshold can vary depending on the situation. In our case, we used a threshold of 5, and all features were below this value except age (VIF ≈ 8.79). So, we trained models with and without age, because the multicollinearity of age affects the logistic regression model in particular. After removing age, all VIF values were under 5.

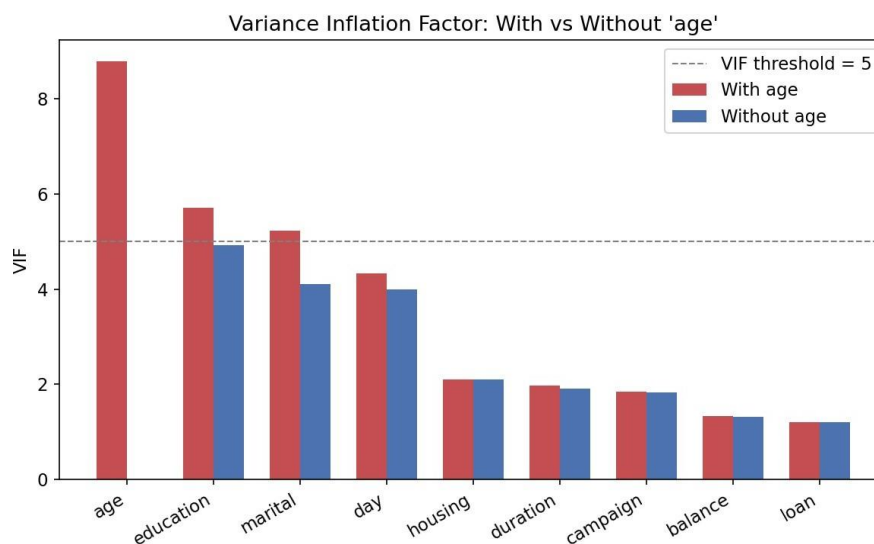


Figure 4. Variance Inflation Factor (VIF) for each feature, with and without age. Age was the only feature above the threshold of 5.

One-Hot Encoding

We applied one-hot encoding using dummy variables to convert categorical columns into binary columns, allowing the model to identify each category without assuming any numerical order between them.

Feature Selection / Feature Importance

For feature selection, we first separated the dependent variable (y) from the independent variables. Then we applied ANOVA (SelectKBest) to pick the top features. As expected, duration came out as the most influential feature, which matches our earlier observation that it strongly affects subscription rates. We then dropped all features scoring below 5. Since day had also been converted to week numbers and was less important, we removed it as well.

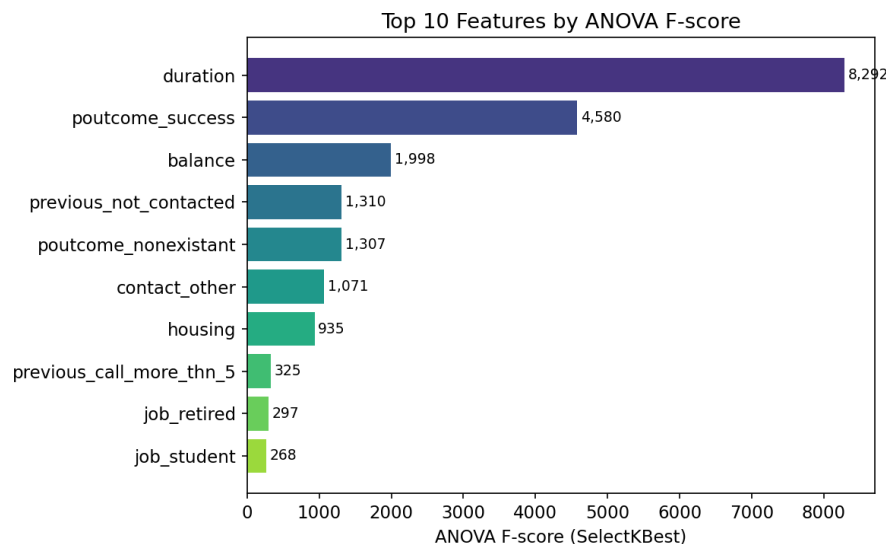


Figure 5. Top 10 features ranked by ANOVA F-score (SelectKBest). duration and poutcome_success dominate the ranking.

Train-Test Split Data

We tried three different train-test splits (70–30, 75–25, and 80–20) and compared their class ratios. We noticed that because the target variable is imbalanced, the model may predict only the majority class ("No") and get high accuracy, while completely failing on the minority class ("Yes"), which is more important for our study than the "No" class. Also, metrics like F1, ROC-AUC, and recall for the "Yes" class would be very low. So we decided to handle the imbalance using oversampling with SMOTETomek before training our models.

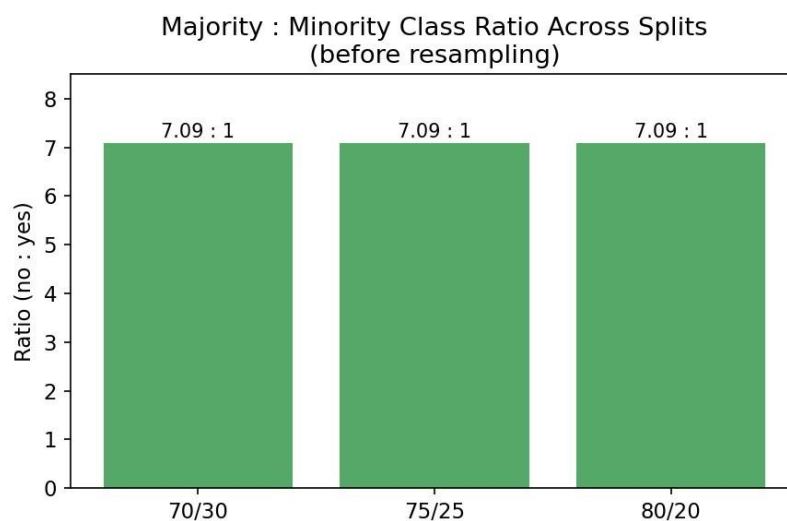


Figure 6. Majority-to-minority class ratio (~7.1 : 1) held consistently across all three train/test splits, before resampling.

Handling Imbalanced Classes (Oversampling)

To fix the imbalance between "Yes" and "No", we used SMOTETomek, which combines two techniques: SMOTE adds synthetic samples to increase the minority class by interpolating between existing minority points and their nearest neighbors, while Tomek Links cleans up confusing majority samples that sit on the decision boundary. This gives us a balanced dataset to train the model properly. After balancing the classes, the next step is to scale the features so the model can read the numerical values properly.

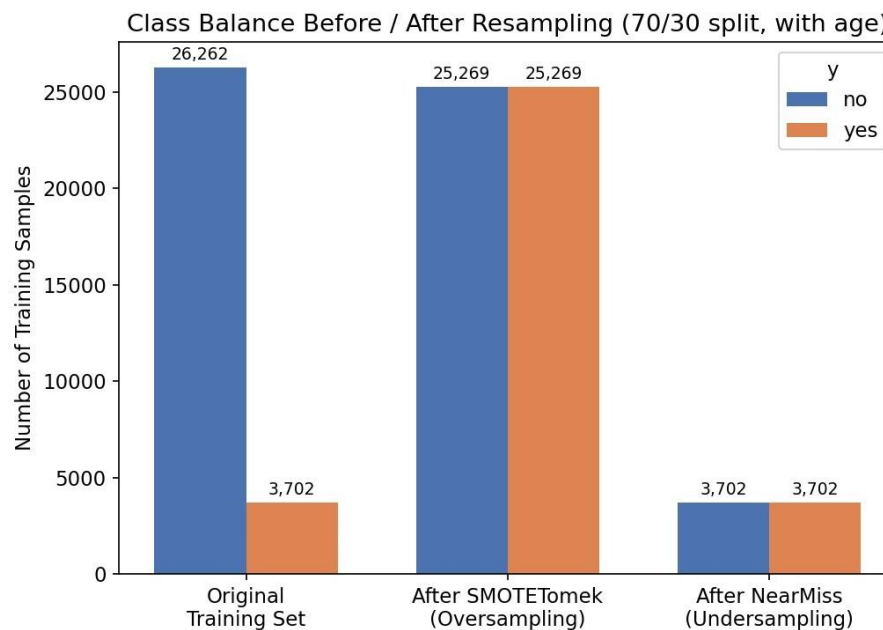


Figure 7. Training-set class counts before resampling, after SMOTETomek (oversampling), and after NearMiss (undersampling) – 70/30 split, with age.

Feature Scaling

After balancing, we scaled all numerical features so they stay on the same range. This helps many classification algorithms learn better and prevents features with large values from affecting the model.

Handling Imbalanced Classes (Undersampling using NearMiss)

NearMiss undersampling was applied as a second strategy to further investigate how reducing the majority class influences model performance. The method removes majority-class samples by selecting those closest to minority-class observations, creating a more compact and informative training set. We applied NearMiss fitting, then split the data into train and test sets, and — same as before — scaled the data. As shown in Figure 7 above, this reduced the training set all the way down to a balanced 3,702 "no" vs 3,702 "yes" (70/30 split, with age) — a much smaller but perfectly balanced training set.

Model Building

For model building, we used five classification algorithms: Logistic Regression, SVM, Random Forest, Gradient Boosting, and XGBoost. We evaluated each model on three different dataset versions (with and without the age column):

- Dataset A → test size 0.30
- Dataset B → test size 0.25
- Dataset C → test size 0.20

Each model was trained and tested on all three splits to compare performance by computing the confusion matrix for each algorithm on each dataset. We also computed ROC curves for every model to visually assess how well each one distinguishes between the "Yes" and "No" classes.

Model Evaluation

After applying SMOTETomek, the predictive models (Logistic Regression, Random Forest, Gradient Boosting, SVM, and XGBoost) demonstrated significant improvements in recall and F1-score, particularly for the minority class. This indicates that the models became more capable of identifying customers who were likely to subscribe to the term deposit. Although accuracy slightly decreased — which is expected in imbalanced problems — the improvement in recall and balanced performance shows that SMOTETomek helped the models generalize better and reduced bias toward the majority class. Across the five models (Dataset A, 70/30 split, with age), accuracy ranged from 86–88% with recall on the "yes" class between 0.56 and 0.67 — overall the most balanced and reliable performance among all resampling strategies.

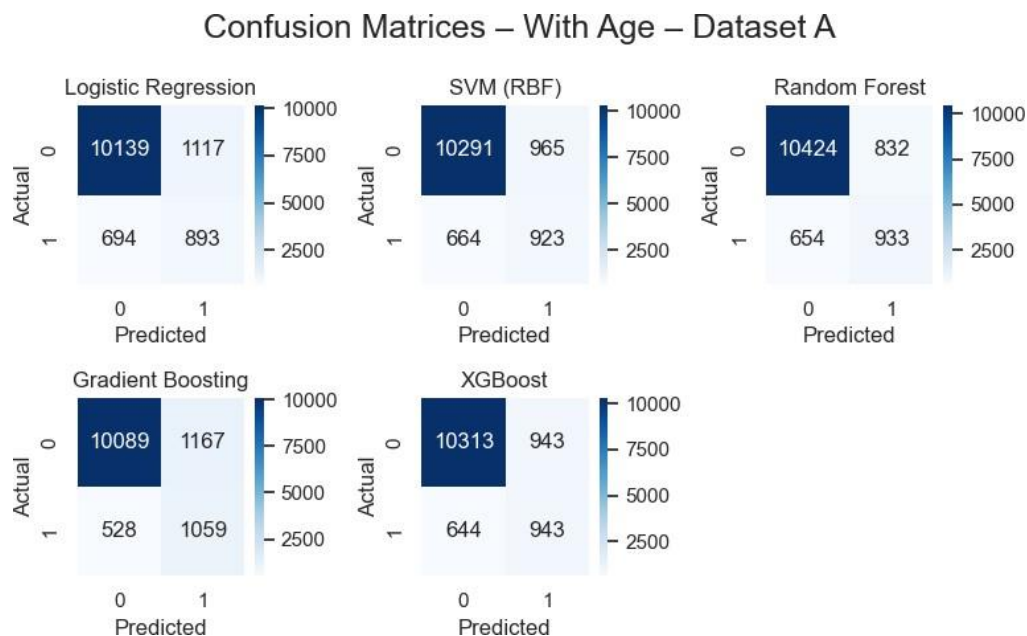


Figure 8. Confusion matrices for all five models SMOTETomek (Dataset A, 70/30 split, with age).

While NearMiss successfully balanced the dataset, its impact on model performance was mixed. Models trained on the undersampled data achieved much higher recall (0.86–0.91) but noticeably lower accuracy (55–75%) and precision, which suggests that the reduction of the majority class led to information loss. This effect was particularly visible in models such as SVM and Logistic Regression, which rely heavily on data distribution. Tree-based models like Random Forest and Gradient Boosting performed better but still showed reduced stability compared to the

SMOTETomek results. Overall, NearMiss helped improve sensitivity to the minority class but introduced increased variance and reduced predictive power.

Confusion Matrices – With Age – Dataset A

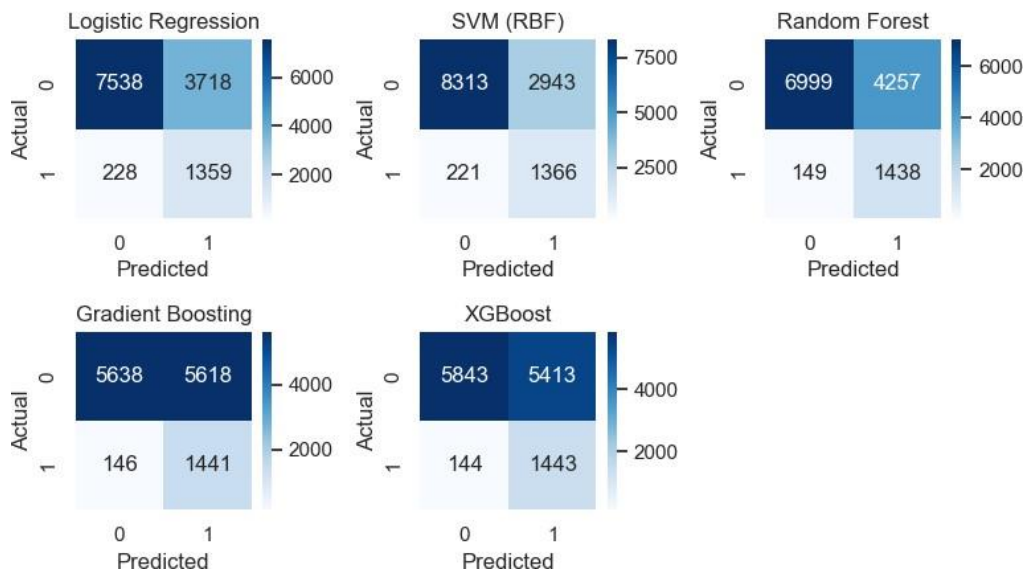


Figure 9. Confusion matrices for all five models NearMiss (Dataset A, 70/30 split, with age).

Comparing accuracy against minority-class recall side by side makes the trade-off clear: SMOTETomek keeps both metrics reasonably balanced, while NearMiss pushes recall much higher at a steep cost to accuracy — most severe for Gradient Boosting and Logistic Regression.

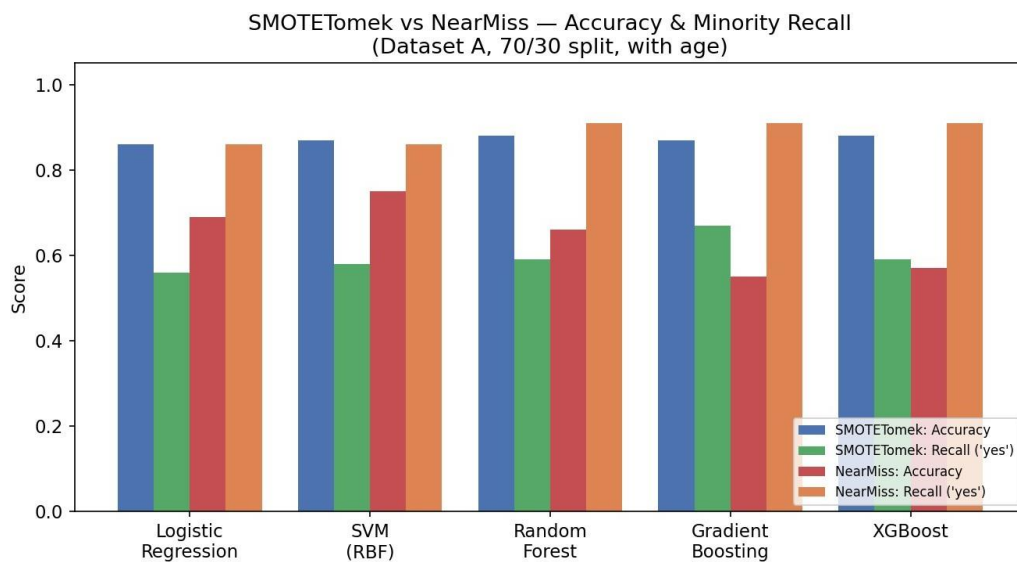


Figure 10. Accuracy vs. recall on the "yes" class, SMOTETomek vs. NearMiss, across all five models.

Challenges

During the course of our project, we faced some challenges and difficulties that made us struggle to find an appropriate solution. Some of these challenges are:

- The target variable ("y") was highly imbalanced, and this created a critical challenge in the project. Handling this issue required using specific techniques so that the model's accuracy and predictions wouldn't be affected.
- Most of the features in the dataset were categorical, which meant we had to do extra work and apply specific techniques to process them. This included applying feature encoding and several additional steps, which required more time and effort.

References

- Dataset: [UCI Machine Learning Repository — Bank Marketing](#)
- [Variance Inflation Factor \(VIF\)](#)
- [SMOTETomek / imbalanced-learn documentation](#)